Course: CS420 – Artificial Intelligence

Midterm examination review – Lectures 0 to 3

PRACTICE MIDTERM EXAMINATION – TEST 2
SOLUTION

Duration: 120 minutes · Closed book · Calculator allowed

Unless a question says otherwise, ties are broken in alphabetical order (left-to-right in trees), and successors are generated in alphabetical order.

Question 1. *Farmer, wolf, goat and cabbage.* A farmer must take a wolf, a goat and a cabbage across a river. The boat holds the farmer and at most one of the three items, and only the farmer can row. Left unattended (without the farmer), the wolf eats the goat and the goat eats the cabbage.

(a) Consider a state as a tuple of values. Which values should be included in the tuple?

(F, W, G, C): the bank (L or R) on which the farmer, the wolf, the goat and the cabbage are. (The boat is always with the farmer.)

(b) Define the initial state and the goal state.

Initial (L, L, L, L); goal (R, R, R, R).

(c) Define the action function.

The farmer crosses alone or with one item that is on *his* bank: Cross, CrossW, CrossG, CrossC. An action is legal only if the result does not leave W with G, or G with C, on the bank without the farmer.

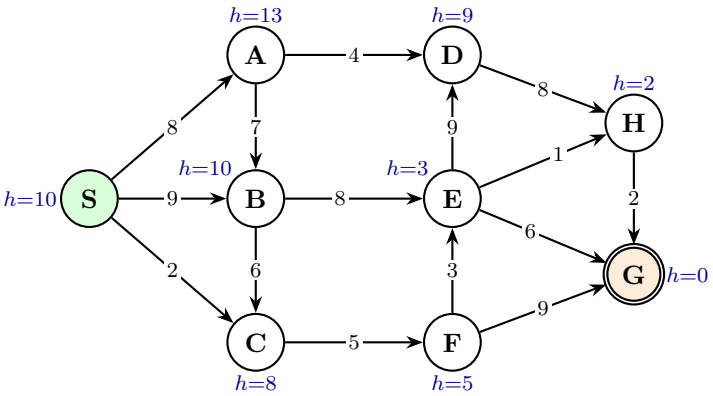
(d) What is the action cost?

1 per crossing.

(e) What is the maximum number of states (including illegal ones)? How many states are legal, and how many are reachable? Give a shortest solution.

$2^4 =$ **16** tuples, **10** legal, **10** reachable. Shortest solution (7 crossings): LLLL → RLRL → LLRL → RRRL → LRLL → RRLR → LRLR → RRRR.

Question 2. Consider the *directed* graph below; S is the initial state and G the goal state.



For each search strategy, (a) list, in order, the states **expanded**, (b) give the path found and (c) give the final content of the frontier (a state is expanded when it is removed from the frontier). Conventions:

BFS, DFS and greedy best-first search test a state for the goal when it is *generated* (so the goal is not expanded); UCS and A* test it when it is *removed* from the frontier (graph search; a cheaper path to a frontier state replaces its entry). DFS avoids loops by remembering the states on the current path only. The path returned will not be accepted if the list of expanded states is wrong.

Algorithm	**List of expanded states**	**Path returned**	**Final frontier**
Breadth-first search	S A B C D E	S B E G (cost 23)	{ F, H }
Uniform-cost search	S C F A B E H D G	S C F E H G (cost 13)	{ }
Depth-first search	S A B C F	S A B C F G (cost 35)	(not asked)
Greedy best-first search	S C F	S C F G (cost 16)	{ E, B, A }
Graph-search A*	S C F E H G	S C F E H G (cost 13)	{ B, A, D }

BFS (queue after each expansion):

Step	Expanded	Queue (front → back) after the expansion
1	S	A, B, C
2	A	B, C, D
3	B	C, D, E
4	C	D, E, F
5	D	E, F, H
6	E	generates goal **G** ⇒ stop; frontier: F, H

DFS (stack after each expansion):

Step	Expanded	Stack (top → bottom) after the expansion
1	S	A, B, C
2	A	B, D, B, C
3	B	C, E, D, B, C
4	C	F, E, D, B, C
5	F	generates goal **G** ⇒ stop; frontier: E, D, B, C

UCS:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $g(n)$)
1	S_0	C_2, A_8, B_9
2	C_2	F_7, A_8, B_9
3	F_7	A_8, B_9, E_{10}, G_{16}
4	A_8	B_9, E_{10}, D_{12}, G_{16}
5	B_9	E_{10}, D_{12}, G_{16}
6	E_{10}	H_{11}, D_{12}, G_{16}
7	H_{11}	D_{12}, G_{13}
8	D_{12}	G_{13}
9	G_{13}	*goal removed from the frontier – stop*

Greedy best-first:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $h(n)$)
1	S_{10}	C_8, B_{10}, A_{13}
2	C_8	F_5, B_{10}, A_{13}
3	F_5	generates goal **G** ⇒ stop; frontier: E_3, B_{10}, A_{13}

A*:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $g + h = f$)
1	$S_{0+10=10}$	$C_{2+8=10}$, $B_{9+10=19}$, $A_{8+13=21}$
2	$C_{2+8=10}$	$F_{7+5=12}$, $B_{9+10=19}$, $A_{8+13=21}$
3	$F_{7+5=12}$	$E_{10+3=13}$, $G_{16+0=16}$, $B_{9+10=19}$, $A_{8+13=21}$
4	$E_{10+3=13}$	$H_{11+2=13}$, $G_{16+0=16}$, $B_{9+10=19}$, $A_{8+13=21}$, $D_{19+9=28}$
5	$H_{11+2=13}$	$G_{13+0=13}$, $B_{9+10=19}$, $A_{8+13=21}$, $D_{19+9=28}$
6	$G_{13+0=13}$	*goal removed from the frontier – stop*

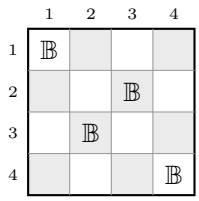
(d) Is h admissible? Is it consistent? Justify (only the direction of each edge matters).

h^*: S= 13, A= 14, B= 11, C= 11, D= 10, E= 3, F= 6, H= 2, G= 0. $h \le h^*$ everywhere ⇒ admissible; $h(u) \le c(u,v) + h(v)$ for every directed edge $u \to v$ ⇒ consistent.

(e) UCS and A* return the same path. Why does A* expand fewer states?

Both are optimal here (cost 13). A* never expands a state with $f(n) > C^*$: A ($f = 8 + 13 = 21$) and B ($f = 9 + 10 = 19$) are never expanded, so A* expands 6 states versus 9 for UCS.

Question 3. *4-bishops.* Consider the 4-bishops problem: every state has 4 bishops on a 4×4 board, one in each column. A bishop attacks along the diagonals (count attacking pairs regardless of pieces in between).



(a) What is the total number of states in the state space? Explain.

$4^4 = \mathbf{256}$: each of the 4 bishops can be in any of the 4 rows of its column.

(b) Each step moves one bishop within its own column. How many successors does a state have? Explain.

$4 \times 3 = \mathbf{12}$: choose the bishop (4 ways) and one of the 3 other rows of its column.

(c) A state is written as 4 digits giving the row of the bishop in columns 1–4 (the board above is 1324). Let n_b be the number of attacking pairs of bishops of state n. Define a fitness function Fit(n) to be maximised by the genetic algorithm.

Fit$(n) = \binom{4}{2} - n_b = 6 - n_b$ (the number of non-attacking pairs); the goal has Fit = 6.

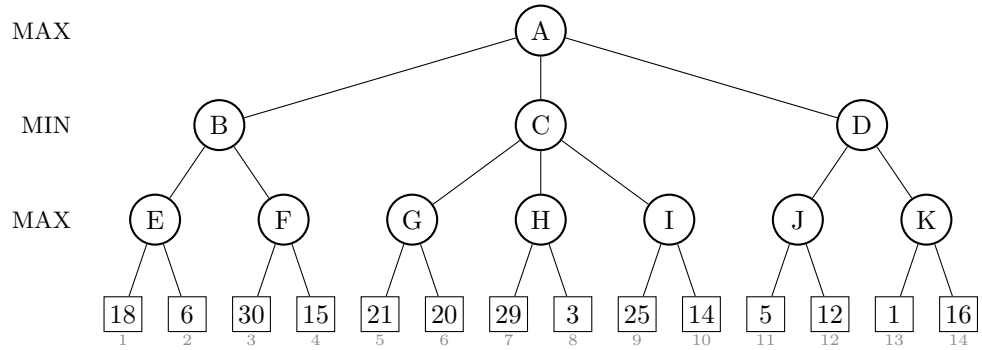
(d) The current generation is $S_1 = 1324$, $S_2 = 2143$, $S_3 = 3412$, $S_4 = 4132$. Compute Fit(n) and the

probability that each state is chosen in the roulette-wheel selection step. Then cross S_1 with S_4 after the 2nd digit and give the two children with their fitness.

State n	$S_1 = 1324$	$S_2 = 2143$	$S_3 = 3412$	$S_4 = 4132$
Fit(n)	4	2	2	5
Prob(n)	$4/13 = 30.8\%$	$2/13 = 15.4\%$	$2/13 = 15.4\%$	$5/13 = 38.5\%$

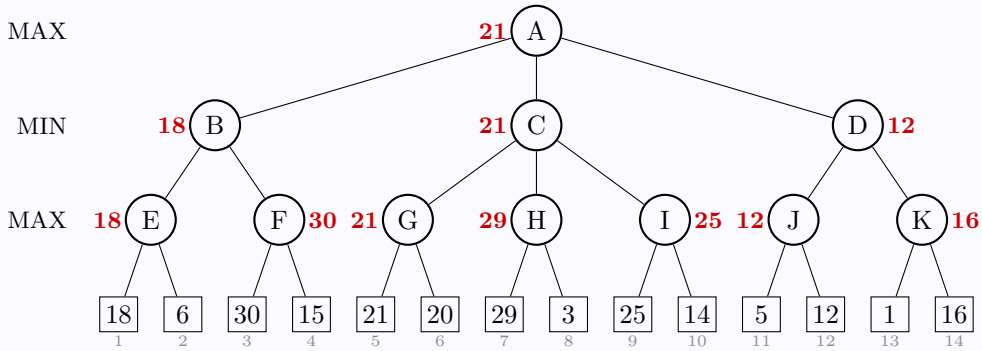
Attacking pairs (by column): S_1: (1,4), (2,3); S_2: (1,2), (1,3), (2,4), (3,4); S_3: (1,2), (1,3), (2,4), (3,4); S_4: (3,4). Children: **1332** (Fit 4) and **4124** (Fit 4).

Question 4. Consider the following game tree. The root is a MAX node; children are visited left to right.



(a) Compute the final backed-up value of every node using the minimax algorithm (no alpha-beta pruning at this step). Which move should MAX make at the root?

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**
Minimax	21	18	21	12	18	30	21	29	25	12	16

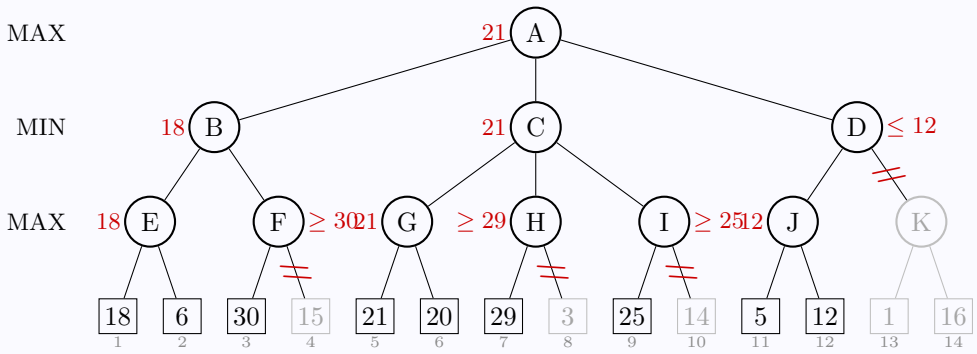


Root value = 21; MAX moves to **C**.

(b) Run alpha-beta pruning (children visited left to right). Give the value computed at every node (mark **X** if the node is never visited), cross out the pruned branches on the tree and briefly explain each case.

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**
α-β	21	18	21	≤ 12	18	≥ 30	21	≥ 29	≥ 25	12	**X**

($\leq v$ / $\geq v$: the node was cut off, so only this bound is known; writing the returned number v is also accepted.)



α and β at every visited node:

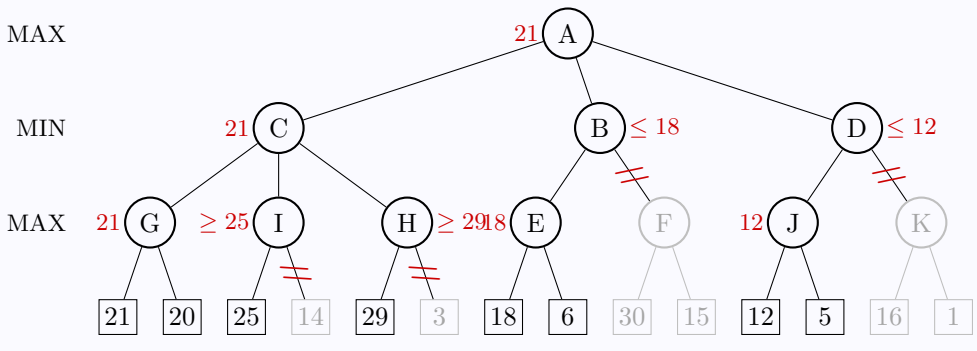
Node	Type	On entry	When it returns	Value returned
A	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 21,\ \beta = +\infty$	21
B	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 18$	18
E	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 18,\ \beta = +\infty$	18
F	MAX	$\alpha = -\infty,\ \beta = 18$	$\alpha = 30,\ \beta = 18$	≥ 30
C	MIN	$\alpha = 18,\ \beta = +\infty$	$\alpha = 18,\ \beta = 21$	21
G	MAX	$\alpha = 18,\ \beta = +\infty$	$\alpha = 21,\ \beta = +\infty$	21
H	MAX	$\alpha = 18,\ \beta = 21$	$\alpha = 29,\ \beta = 21$	≥ 29
I	MAX	$\alpha = 18,\ \beta = 21$	$\alpha = 25,\ \beta = 21$	≥ 25
D	MIN	$\alpha = 21,\ \beta = +\infty$	$\alpha = 21,\ \beta = 12$	≤ 12
J	MAX	$\alpha = 21,\ \beta = +\infty$	$\alpha = 21,\ \beta = +\infty$	12

- At MAX node **F**, after leaf #3: $v = 30 \geq \beta = 18 \Rightarrow$ prune leaf #4. (MIN at B already has an alternative worth $\beta = 18$, so it will never let play reach F.)
- At MAX node **H**, after leaf #7: $v = 29 \geq \beta = 21 \Rightarrow$ prune leaf #8. (MIN at C already has an alternative worth $\beta = 21$, so it will never let play reach H.)
- At MAX node **I**, after leaf #9: $v = 25 \geq \beta = 21 \Rightarrow$ prune leaf #10. (MIN at C already has an alternative worth $\beta = 21$, so it will never let play reach I.)
- At MIN node **D**, after child J: $v = 12 \leq \alpha = 21 \Rightarrow$ prune K (leaves #13–#14). (MAX at A already has an alternative worth $\alpha = 21$, so it will never let play reach D.)

Leaves evaluated: 9 of 14.

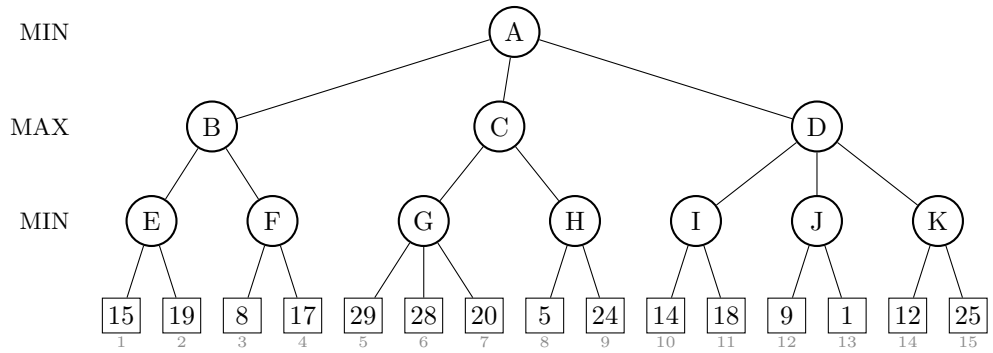
(c) Using the minimax values from (a), and without performing any alpha-beta calculation, rotate the children of each node at every level so that alpha-beta prunes as much as possible. Draw the new tree (keep the node letters).

Best child first for the player to move (MAX: largest value first, MIN: smallest first):



With this ordering alpha-beta evaluates only **8** leaves (instead of 9).

Question 5. Consider the game tree below, in which **MIN moves first** (the root is a MIN node).

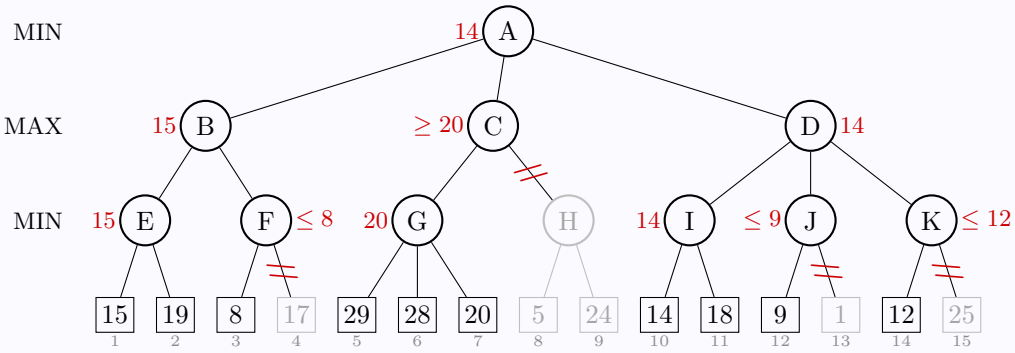


(a) Note the minimax value of every node (no pruning).

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**
Minimax	14	15	20	14	15	8	20	5	14	1	12

Root value = 14; MIN moves to **D**.

(b) Show where alpha-beta pruning occurs (left to right) by crossing out the corresponding branches, and briefly explain each case.



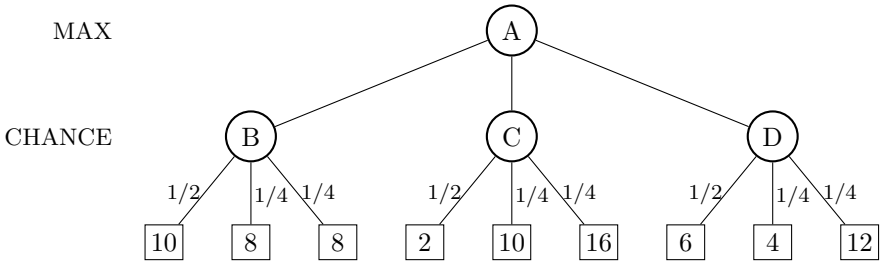
Node	Type	On entry	When it returns	Value returned
A	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 14$	14
B	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 15,\ \beta = +\infty$	15
E	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 15$	15
F	MIN	$\alpha = 15,\ \beta = +\infty$	$\alpha = 15,\ \beta = 8$	≤ 8
C	MAX	$\alpha = -\infty,\ \beta = 15$	$\alpha = 20,\ \beta = 15$	≥ 20
G	MIN	$\alpha = -\infty,\ \beta = 15$	$\alpha = -\infty,\ \beta = 15$	20
D	MAX	$\alpha = -\infty,\ \beta = 15$	$\alpha = 14,\ \beta = 15$	14
I	MIN	$\alpha = -\infty,\ \beta = 15$	$\alpha = -\infty,\ \beta = 14$	14
J	MIN	$\alpha = 14,\ \beta = 15$	$\alpha = 14,\ \beta = 9$	≤ 9
K	MIN	$\alpha = 14,\ \beta = 15$	$\alpha = 14,\ \beta = 12$	≤ 12

- At MIN node **F**, after leaf #3: $v = 8 \leq \alpha = 15 \Rightarrow$ prune leaf #4. (MAX at B already has an alternative worth $\alpha = 15$, so it will never let play reach F.)
- At MAX node **C**, after child G: $v = 20 \geq \beta = 15 \Rightarrow$ prune H (leaves #8–#9). (MIN at A already has an alternative worth $\beta = 15$, so it will never let play reach C.)

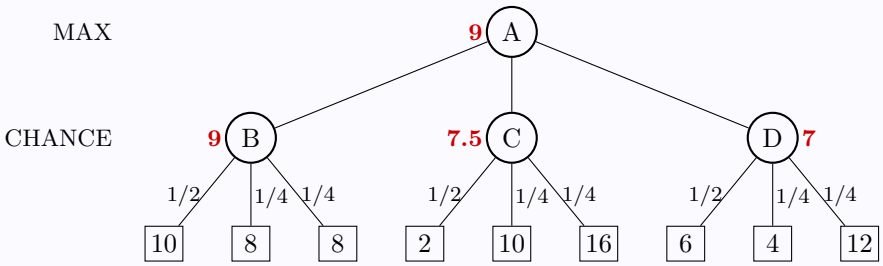
- At MIN node **J**, after leaf #12: $v = 9 \leq \alpha = 14 \Rightarrow$ prune leaf #13. (MAX at D already has an alternative worth $\alpha = 14$, so it will never let play reach J.)
- At MIN node **K**, after leaf #14: $v = 12 \leq \alpha = 14 \Rightarrow$ prune leaf #15. (MAX at D already has an alternative worth $\alpha = 14$, so it will never let play reach K.)

Leaves evaluated: 10 of 15; never evaluated: #4, #8, #9, #13, #15.

Question 6. Consider the following game tree with chance nodes (outcome probabilities on the edges).



(a) Calculate the expectimax value of every node. Which move does MAX choose?



B = 1/2·10+1/4·8+1/4·8 = 9; C = 1/2·2+1/4·10+1/4·16 = 7.5; D = 1/2·6+1/4·4+1/4·12 = 7. MAX chooses **B** (9).

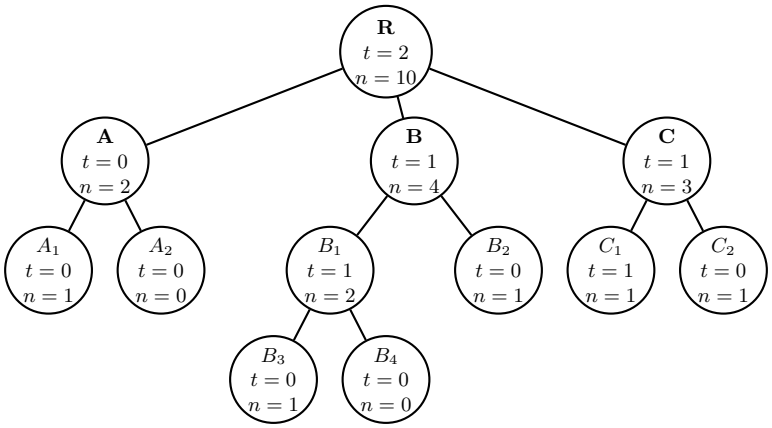
(b) Which move would MAX choose if the chance nodes were MIN nodes?

B= 8, C= 2, D= 4 ⇒ B.

(c) Replace every leaf value x by x^2 (order-preserving for these positive values). Does the expectimax decision change? What does this tell you about evaluation functions in stochastic games?

B = 82, C = 91, D = 58 ⇒ MAX now chooses **C**: the decision changes. For expectimax the magnitudes of the values matter (only a positive linear transformation is safe), whereas minimax only needs the order.

Question 7. Consider the MCTS tree below; rollouts return 1 (win for the root player) or 0 (loss), t is the total reward and n the number of visits. Use UCB1 = $\bar{V}_i + C\sqrt{\ln N / n_i}$ with $C = \sqrt{2}$ (unvisited = ∞, ties to the left). A leaf never visited is rolled out directly; otherwise all its children are added and the rollout starts at the first. Expansions create: $A_1 \to A_3, A_4$; $A_2 \to A_5, A_6$; $B_1 \to B_3, B_4$; $B_2 \to B_5, B_6$; $C_1 \to C_3, C_4$; $C_2 \to C_5, C_6$.



(a) Calculate UCB1 for A, B and C. Which child is selected?

A: $\frac{0}{2} + \sqrt{2}\sqrt{\frac{\ln 10}{2}} = 1.517$; B: $\frac{1}{4} + \sqrt{2}\sqrt{\frac{\ln 10}{4}} = 1.323$; C: $\frac{1}{3} + \sqrt{2}\sqrt{\frac{\ln 10}{3}} = 1.572 \Rightarrow$ **C**.

(b) Perform three iterations whose rollouts return 1, 0, 0 (in order); update the statistics and draw the final tree.

Iteration 1.

- *Selection* at R ($N = 10$): A: $\frac{0}{2} + \sqrt{2}\sqrt{\frac{\ln 10}{2}} = 1.517$; B: $\frac{1}{4} + \sqrt{2}\sqrt{\frac{\ln 10}{4}} = 1.323$; C: $\frac{1}{3} + \sqrt{2}\sqrt{\frac{\ln 10}{3}} = 1.572 \Rightarrow$ go to **C**.
- *Selection* at C ($N = 3$): C_1: $\frac{1}{1} + \sqrt{2}\sqrt{\frac{\ln 3}{1}} = 2.482$; C_2: $\frac{0}{1} + \sqrt{2}\sqrt{\frac{\ln 3}{1}} = 1.482 \Rightarrow$ go to C_1.
- *Expansion*: C_1 was visited before ($n > 0$) and has no children $\Rightarrow$ add C_3, C_4; roll out from the first one, C_3.
- *Simulation*: rollout from C_3 returns 1.
- *Backpropagation*: add 1 to t and 1 to n for R $\rightarrow$ C $\rightarrow C_1 \rightarrow C_3$.

Statistics (t/n) after iteration 1: R: 3/11, A: 0/2, A_1: 0/1, A_2: 0/0, B: 1/4, B_1: 1/2, B_3: 0/1, B_4: 0/0, B_2: 0/1, C: 2/4, C_1: 2/2, C_3: 1/1, C_4: 0/0, C_2: 0/1.

Iteration 2.

- *Selection* at R ($N = 11$): A: $\frac{0}{2} + \sqrt{2}\sqrt{\frac{\ln 11}{2}} = 1.549$; B: $\frac{1}{4} + \sqrt{2}\sqrt{\frac{\ln 11}{4}} = 1.345$; C: $\frac{2}{4} + \sqrt{2}\sqrt{\frac{\ln 11}{4}} = 1.595 \Rightarrow$ go to **C**.
- *Selection* at C ($N = 4$): C_1: $\frac{2}{2} + \sqrt{2}\sqrt{\frac{\ln 4}{2}} = 2.177$; C_2: $\frac{0}{1} + \sqrt{2}\sqrt{\frac{\ln 4}{1}} = 1.665 \Rightarrow$ go to C_1.
- *Selection* at C_1 ($N = 2$): C_3: $\frac{1}{1} + \sqrt{2}\sqrt{\frac{\ln 2}{1}} = 2.177$; C_4: ∞ (unvisited) $\Rightarrow$ go to C_4.
- *Expansion*: none — C_4 has $n = 0$, so we roll out directly from it.
- *Simulation*: rollout from C_4 returns 0.
- *Backpropagation*: add 0 to t and 1 to n for R $\rightarrow$ C $\rightarrow C_1 \rightarrow C_4$.

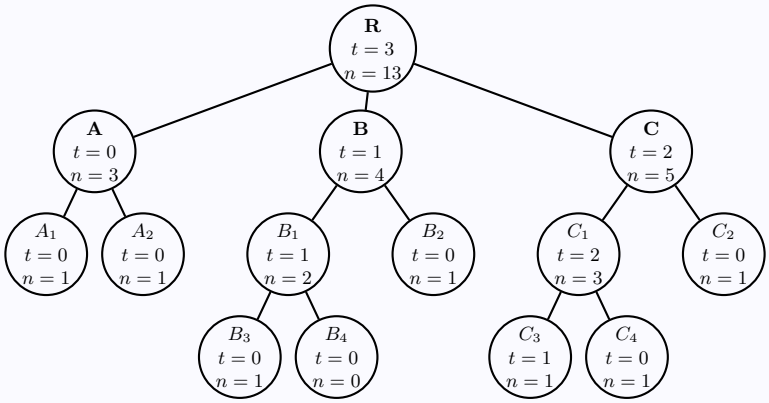
Statistics (t/n) after iteration 2: R: 3/12, A: 0/2, A_1: 0/1, A_2: 0/0, B: 1/4, B_1: 1/2, B_3: 0/1, B_4: 0/0, B_2: 0/1, C: 2/5, C_1: 2/3, C_3: 1/1, C_4: 0/1, C_2: 0/1.

Iteration 3.

- *Selection* at R ($N = 12$): A: $\frac{0}{2} + \sqrt{2}\sqrt{\frac{\ln 12}{2}} = 1.576$; B: $\frac{1}{4} + \sqrt{2}\sqrt{\frac{\ln 12}{4}} = 1.365$; C: $\frac{2}{5} + \sqrt{2}\sqrt{\frac{\ln 12}{5}} = 1.397 \Rightarrow$ go to **A**.
- *Selection* at A ($N = 2$): A_1: $\frac{0}{1} + \sqrt{2}\sqrt{\frac{\ln 2}{1}} = 1.177$; A_2: ∞ (unvisited) $\Rightarrow$ go to A_2.
- *Expansion*: none — A_2 has $n = 0$, so we roll out directly from it.

- *Simulation*: rollout from A_2 returns 0.
- *Backpropagation*: add 0 to t and 1 to n for R → A → A_2.

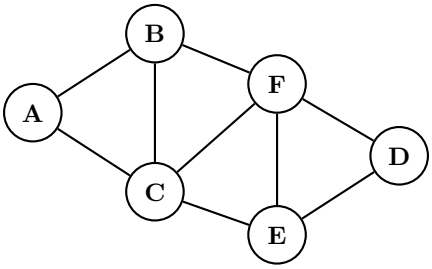
Statistics (t/n) after iteration 3: R: 3/13, A: 0/3, A_1: 0/1, A_2: 0/1, B: 1/4, B_1: 1/2, B_3: 0/1, B_4: 0/0, B_2: 0/1, C: 2/5, C_1: 2/3, C_3: 1/1, C_4: 0/1, C_2: 0/1.
Final tree:



(c) If the search stopped now, which move is played by the robust-child rule (the lecture's `best_child`)? Does the max-child rule agree?

A: 0/3, B: 1/4, C: 2/5 ⇒ robust child **C**; max child **C**.

Question 8. This problem is about map colouring. Each region must be coloured Red (R), Green (G) or Blue (B), and neighbouring regions must have different colours. The constraint graph is shown below. The following questions are mutually independent.



(a) Cross out all values that would be eliminated by *forward checking* after variable B has been assigned as shown.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	R G B	**R**	R G B	R G B	R G B	R G B

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	~~R~~ G B	**R** ~~G B~~	~~R~~ G B	R G B	R G B	~~R~~ G B

Only the neighbours of B (A, C, F) lose R.

(b) A and B have been assigned as shown, but no constraint propagation has been done yet. **Cross out** all values that would be eliminated by *arc consistency* (AC-3).

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	**G**	R G B	R G B	R G B	R G B

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R** ~~G~~ ~~B~~	~~R~~ **G** ~~B~~	~~R~~ ~~G~~ B	~~R~~ ~~G~~ B	~~R~~ G ~~B~~	R ~~G~~ ~~B~~

Propagation continues through the graph (unlike forward checking): every arc $X_i \to X_j$ is revised until no domain changes. Here arc consistency even leaves a single value for every variable.

(c) Variable A is already assigned and constraint propagation has been done. **Circle** all unassigned variables that might be selected by the *Minimum-Remaining-Values* (MRV) heuristic.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	G B	G B	R G B	R G B	R G B

Remaining values: B: 2, C: 2, D: 3, E: 3, F: 3 ⇒ MRV may select **B, C**.

(d) Same situation. **Circle** all unassigned variables that might be selected by the *Degree Heuristic*.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	G B	G B	R G B	R G B	R G B

Degree = number of constraints with other *unassigned* variables: B: 2, C: 3, D: 2, E: 3, F: 4 ⇒ the degree heuristic may select **F**.

(e) Assume no variable has been assigned yet. Solve the CSP using backtracking with *forward checking*, choosing variables with MRV, then the degree heuristic (DH), and finally alphabetical order; try values in the order R, G, B. For every step give the MRV value of every region not yet coloured, and the DH values of the regions tied on the minimum MRV.

Step 1

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	3	3	3	3	3	3
DH	2	3	4	2	3	4

Assign to the variable **C** with the value **R** (tie on MRV and DH, broken alphabetically). Domains before this step: A=RGB, B=RGB, C=RGB, D=RGB, E=RGB, F=RGB.

Step 2

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	2	2	–	3	2	2
DH	1	2			2	3

Assign to the variable **F** with the value **G** (tie on MRV broken by DH). Domains before this step: A=GB, B=GB, C=R, D=RGB, E=GB, F=GB.

Step 3

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	2	1	–	2	1	–
DH		1			1	

Assign to the variable **B** with the value **B** (tie on MRV and DH, broken alphabetically). Domains before this step: A=GB, B=B, C=R, D=RB, E=B, F=G.

Step 4

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	1	–	–	2	1	–
DH	0				1	

Assign to the variable **E** with the value **B** (tie on MRV broken by DH). Domains before this step: A=G, B=B, C=R, D=RB, E=B, F=G.

Step 5

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	1	–	–	1	–	–
DH	0			0		

Assign to the variable **A** with the value **G** (tie on MRV and DH, broken alphabetically). Domains before this step: A=G, B=B, C=R, D=R, E=B, F=G.

Step 6

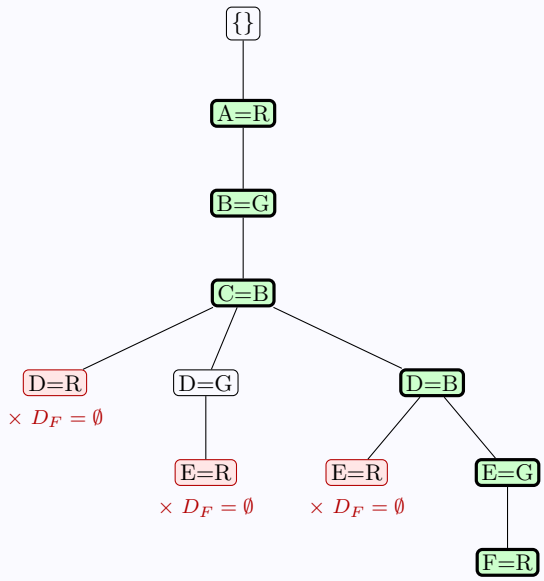
Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	–	–	–	1	–	–
DH						

Assign to the variable **D** with the value **R** (unique minimum MRV). Domains before this step: A=G, B=B, C=R, D=R, E=B, F=G.

Solution: A=G, B=B, C=R, D=R, E=B, F=G. No backtracking is needed.

(f) Now solve the CSP again with forward checking but *without* MRV/DH: variables in the fixed order A, B, C, D, E, F and values in the order R, G, B. Draw the search tree and mark every node where forward checking empties a domain.

Step	Assign	A	B	C	D	E	F	Note
1	A=R	**R**	GB	GB	RGB	RGB	RGB	
2	B=G	*R*	**G**	B	RGB	RGB	RB	
3	C=B	*R*	*G*	**B**	RGB	RG	R	
4	D=R	*R*	*G*	*B*	**R**	G	$\emptyset$	D_F wiped out
backtrack: undo D=R								
5	D=G	*R*	*G*	*B*	**G**	R	R	
6	E=R	*R*	*G*	*B*	*G*	**R**	$\emptyset$	D_F wiped out
backtrack: undo E=R								
backtrack: undo D=G								
7	D=B	*R*	*G*	*B*	**B**	RG	R	
8	E=R	*R*	*G*	*B*	*B*	**R**	$\emptyset$	D_F wiped out
backtrack: undo E=R								
9	E=G	*R*	*G*	*B*	*B*	**G**	R	
10	F=R	*R*	*G*	*B*	*B*	*G*	**R**	



Forward checking empties a domain 3 times and the search backtracks 4 times (solution A=R, B=G, C=B, D=B, E=G, F=R). After $D = G$, FC leaves $E = \{R\}$ and $F = \{R\}$ although E and F are adjacent: FC does not check arcs between unassigned variables, while arc consistency would.

Question 9. Variables $A, B, C \in \{1, \ldots, 6\}$ with the constraints $A = 2B$, $B + C = 7$ and $A < C$.

(a) Run AC-3 starting from the queue $A \to B$, $B \to A$, $B \to C$, $C \to B$, $A \to C$, $C \to A$ (pop from the front; when D_i shrinks, append every arc $X_k \to X_i$, $k \neq j$, not already queued). Show every revision.

#	Arc popped	Removed from tail	New domain	Arcs appended to queue
1	$A \to B$	1, 3, 5	$D_A = \{2, 4, 6\}$	–
2	$B \to A$	4, 5, 6	$D_B = \{1, 2, 3\}$	–
3	$B \to C$	–		–
4	$C \to B$	1, 2, 3	$D_C = \{4, 5, 6\}$	–
5	$A \to C$	6	$D_A = \{2, 4\}$	$B \to A$
6	$C \to A$	–		–
7	$B \to A$	3	$D_B = \{1, 2\}$	$C \to B$
8	$C \to B$	4	$D_C = \{5, 6\}$	$A \to C$
9	$A \to C$	–		–

(b) Cross out the eliminated values, and give all solutions.

Variables	**A**	**B**	**C**
Domains	1 2 3 4 5 6	1 2 3 4 5 6	1 2 3 4 5 6

Variables	**A**	**B**	**C**
Domains	~~1~~ 2 ~~3~~ 4 ~~5~~ ~~6~~	1 2 ~~3~~ ~~4~~ ~~5~~ ~~6~~	~~1~~ ~~2~~ ~~3~~ ~~4~~ 5 6

Solutions (A, B, C): $(2, 1, 6)$, $(4, 2, 5)$.

Question 10. Answer the following questions briefly.

(a) What is a rational agent? Using the lecture's examples (fingers on a hot stove; crossing at a crosswalk), distinguish a reflex action from an intelligent action. Can a reflex action be rational?

A rational agent acts to achieve the best outcome, or the best expected outcome under uncertainty. A reflex action is immediate, without deliberation (withdrawing the fingers); an intelligent action results from reasoning or learning (crossing at the crosswalk). A reflex action can be rational or not; intelligent actions are usually rational.

(b) What is abstraction in problem formulation? When is an abstraction valid and when is it useful? Illustrate with route finding in Romania.

Removing detail from the representation (the state is just the current city; weather, traffic, companions are ignored). Valid: every abstract solution can be turned into a real one; useful: each abstract action (drive Arad→Sibiu) is easier than the original problem.

(c) For 8-queens, compare the naive incremental formulation, the improved incremental formulation and the complete-state formulation used by hill climbing (size of the space / number of successors).

Naive (add a queen to any empty square): $64 \cdot 63 \cdots 57 \approx 1.8 \times 10^{14}$ sequences. Improved (one queen per column, leftmost empty column, non-attacked squares only): 2,057 states. Complete-state (8 queens, one per column, move a queen in its column): $8 \times 7 = 56$ successors per state.

(d) List the four steps of an MCTS iteration and explain the difference between the tree policy and the default policy.

Selection, Expansion, Simulation (rollout), Backpropagation. The tree policy (e.g. UCB1) chooses nodes inside the tree; the default policy plays the rollout outside the tree (uniformly random and fast by default).

Question 11. True or false? For each statement, circle **T** or **F** and justify your answer in one or two sentences (an answer without a correct justification earns no credit).

1. Tree-search DFS without any cycle checking is complete on every finite state space. **T** **F**

FALSE. With cycles (e.g. two states linked both ways) it can loop forever; checking the current path fixes this.

2. Uniform-cost search is complete if every action cost is at least some $\epsilon > 0$. **T** **F**

TRUE. Then any path cost grows without bound with its length, so UCS cannot get stuck on an infinite path of tiny costs.

3. Tree-search DFS needs only $O(bm)$ memory. **T** **F**

TRUE. It stores one path and the unexpanded siblings along it.

4. Greedy best-first graph search is complete in finite state spaces. **T** **F**

TRUE. It never re-expands a state, so it eventually exhausts a finite space.

5. Weighted A* ($f = g + W \cdot h$, $W > 1$) always returns an optimal path when h is admissible. **T** **F**

FALSE. $W \cdot h$ may overestimate; weighted A* trades optimality for speed.

6. Minimax has time complexity $O(b^m)$ and space complexity $O(bm)$. **T** **F**

TRUE. It is a complete depth-first exploration of the game tree.

7. An order-preserving transformation of the leaf values never changes the move chosen by expectimax. **T** **F**

FALSE. True for minimax, not for expectimax (see Question 6).

8. The robust child of the MCTS root is the child with the highest average reward. **T** **F**

FALSE. Robust = most visited; highest average = max child.

9. AC-3 runs in $O(cd^3)$ in the worst case. **T** **F**

TRUE. Each arc is re-queued at most d times and each revision costs $O(d^2)$.

10. Forward checking makes the neighbours of the just-assigned variable consistent with it, but does not look further ahead. **T** **F**

TRUE. That is exactly FC; MAC continues propagating with AC-3.